

MANUAL DE EJECUCIÓN — NAVI HERRAMIENTAS & ACTIVOS

Instalación y arranque desde otro equipo en MacBook Pro y Windows

1. Objetivo

Este manual explica cómo ejecutar el proyecto **NAVI Herramientas & Activos** desde otro equipo, usando el código fuente desde Git, Docker, SQL Server, MinIO y el backup de base de datos NaviToolsAssetsDb.bak.

El objetivo es que cualquier equipo nuevo pueda levantar:

SQL Server Docker

MinIO Docker

Base de datos NaviToolsAssetsDb restaurada desde backup

API .NET

Admin Web Blazor

Mobile PWA

2. Puertos estándar del proyecto

Para evitar inconsistencias, este manual deja el proyecto con estos puertos:

SQL Server: localhost,1433

MinIO API: http://localhost:9000

MinIO Console: http://localhost:9001

API: http://localhost:5218

Admin Web: http://localhost:5264

Mobile PWA: http://localhost:5285

Swagger: http://localhost:5218/swagger

3. Archivos que debes tener antes de iniciar

En el equipo nuevo debes tener:

1. Código fuente del proyecto en Git.

2. URL del repositorio Git.

3. Archivo backup de SQL Server:
backups/NaviToolsAssetsDb.bak

4. Si aplica, backup de documentos MinIO:
backups/minio/navi-tools-documents

5. Credenciales locales para:
SQL Server
MinIO

Estructura esperada del proyecto:

Herramientas-Activos/

Navitrans.ToolsAssets.Management.sln

global.json

```
README.md
docker/
src/
scripts/
docs/
tests/
backups/
  NaviToolsAssetsDb.bak
```

4. Reglas importantes

4.1 No ejecutar el archivo .bak

El archivo:

```
backups/NaviToolsAssetsDb.bak
```

no se ejecuta con doble clic ni escribiendo su ruta en consola.

Correcto:

Se copia al contenedor SQL Server y se restaura con RESTORE DATABASE.

4.2 No subir contraseñas a Git

No subir:

```
docker/env/local.env
```

```
backups/*.bak
```

archivos reales de MinIO

appsettings con passwords reales

4.3 No trabajar como root en Mac

En Mac, trabajar como usuario normal, por ejemplo:

```
luu
```

No usar una terminal como root para Docker Desktop.

5. PARTE A — Crear backup desde el equipo origen

Esta sección se usa en el equipo donde ya tienes la base funcionando.

5.1 Crear backup en Mac

Ubícate en la raíz del proyecto:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-Activos
```

Ejecuta:

```
mkdir -p backups
```

```
export MSSQL_SA_PASSWORD="$(grep '^MSSQL_SA_PASSWORD=' docker/
env/local.env | cut -d= -f2-)"
```

```

docker exec navi-tools-sqlserver mkdir -p /var/opt/mssql/backup

SQLCMD="/opt/mssql-tools18/bin/sqlcmd"

if ! docker exec navi-tools-sqlserver test -x "$SQLCMD"; then
    SQLCMD="/opt/mssql-tools/bin/sqlcmd"
fi

docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD" navi-tools-
sqlserver "$SQLCMD" \
    -S localhost \
    -U sa \
    -C \
    -Q "BACKUP DATABASE [NaviToolsAssetsDb] TO DISK = N'/var/opt/
mssql/backup/NaviToolsAssetsDb.bak' WITH INIT, COMPRESSION,
STATS = 10;"

docker cp navi-tools-sqlserver:/var/opt/mssql/backup/
NaviToolsAssetsDb.bak backups/NaviToolsAssetsDb.bak

ls -lh backups/NaviToolsAssetsDb.bak

```

5.2 Crear backup en Windows PowerShell

Ubícate en la raíz del proyecto:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

Ejecuta:

```
New-Item -ItemType Directory -Force -Path ".\backups" | Out-
Null
```

```
$SqlPass = (Get-Content ".\docker\env\local.env" | Where-Object
{ $_ -like "MSSQL_SA_PASSWORD=*" }) -replace
"^MSSQL_SA_PASSWORD=", ""
```

```
docker exec navi-tools-sqlserver mkdir -p /var/opt/mssql/backup
```

```
$SqlCmd = "/opt/mssql-tools18/bin/sqlcmd"
```

```
docker exec navi-tools-sqlserver test -x $SqlCmd
```

```
if ($LASTEXITCODE -ne 0) {
    $SqlCmd = "/opt/mssql-tools/bin/sqlcmd"
}
```

```
docker exec -e SQLCMDPASSWORD="$SqlPass" navi-tools-sqlserver
```

```
$SqlCmd `
    -S localhost `
    -U sa `
    -C `
    -Q "BACKUP DATABASE [NaviToolsAssetsDb] TO DISK = N'/var/
opt/mssql/backup/NaviToolsAssetsDb.bak' WITH INIT, COMPRESSION,
STATS = 10;"
```

```
docker cp navi-tools-sqlserver:/var/opt/mssql/backup/
NaviToolsAssetsDb.bak ".\backups\NaviToolsAssetsDb.bak"
```

```
Get-Item ".\backups\NaviToolsAssetsDb.bak"
```

6. PARTE B — Instalación en MacBook Pro

6.1 Instalar herramientas base en Mac

Abrir Terminal y ejecutar:

```
xcode-select --install
```

Instalar Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/
Homebrew/install/HEAD/install.sh)"
```

Agregar Homebrew al PATH si es Apple Silicon:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Agregar Homebrew al PATH si es Intel:

```
echo 'eval "$(/usr/local/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/usr/local/bin/brew shellenv)"
```

Verificar:

```
brew --version
```

Instalar Git, wget y VS Code:

```
brew update
```

```
brew install git wget
```

```
brew install --cask visual-studio-code || true
```

Abrir VS Code:

```
open -a "Visual Studio Code"
```

6.2 Instalar Docker Desktop en Mac

```
brew install --cask docker || true
```

```
open -a Docker
```

Verificar:

```
docker --version
```

```
docker compose version
```

```
docker info
```

Si tu Mac es Apple Silicon, instala Rosetta:

```
softwareupdate --install-rosetta --agree-to-license || true
```

Verificar arquitectura:

```
uname -m
```

Resultado esperado:

```
arm64    para Apple Silicon
```

```
x86_64   para Intel
```

6.3 Instalar .NET 8 SDK en Mac

Ejecuta:

```
rm -f /tmp/dotnet-install.sh
```

```
curl -sSL https://dot.net/v1/dotnet-install.sh -o /tmp/dotnet-  
install.sh
```

```
chmod +x /tmp/dotnet-install.sh
```

```
/tmp/dotnet-install.sh --channel 8.0 --install-dir  
"$HOME/.dotnet"
```

Agregar .NET al PATH:

```
cat >> ~/.zshrc <<'EOF'
```

```
# .NET SDK local
```

```
export DOTNET_ROOT="$HOME/.dotnet"
```

```
export PATH="$PATH:$DOTNET_ROOT:$DOTNET_ROOT/tools"
```

```
EOF
```

```
source ~/.zshrc
```

Verificar:

```
dotnet --list-sdks
```

```
dotnet --version
```

Instalar Entity Framework:

```
dotnet tool install --global dotnet-ef --version 8.0.20 ||
```

```
dotnet tool update --global dotnet-ef --version 8.0.20
```

Verificar:

```
dotnet ef --version
```

6.4 Clonar proyecto en Mac

Crear carpeta:

```
mkdir -p /Users/luu/Desktop/Proyecto_Navi/Projects
```

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects
```

Clonar:

```
git clone PEGAR_URL_DEL_REPOSITORIO_GIT Herramientas-Activos
```

Entrar:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
```

Activos
Verificar:
pwd
ls
git status

6.5 Copiar backup al proyecto en Mac

Crear carpeta:
mkdir -p backups
Copiar el archivo:
cp /RUTA/DONDE/TENGAS/NaviToolsAssetsDb.bak backups/
NaviToolsAssetsDb.bak
Validar:
ls -lh backups/NaviToolsAssetsDb.bak

6.6 Crear archivo de variables local.env en Mac

Ejecuta:
mkdir -p docker/env

cat > docker/env/local.env <<'EOF'
ACCEPT_EULA=Y
MSSQL_SA_PASSWORD=PEGAR_PASSWORD_SQL_LOCAL
MSSQL_PID=Developer
MSSQL_DB=NaviToolsAssetsDb

MINIO_ROOT_USER=PEGAR_USUARIO_MINIO_LOCAL
MINIO_ROOT_PASSWORD=PEGAR_PASSWORD_MINIO_LOCAL
MINIO_BUCKET=navi-tools-documents
EOF

chmod 600 docker/env/local.env
Verificar sin compartir capturas:
cat docker/env/local.env

6.7 Crear docker-compose.yml estándar en Mac

Ejecuta:
cat > docker/docker-compose.yml <<'EOF'
name: navi-tools-assets

services:
 sqlserver:
 image: mcr.microsoft.com/mssql/server:2022-latest
 container_name: navi-tools-sqlserver
 restart: unless-stopped

```

environment:
  ACCEPT_EULA: ${ACCEPT_EULA}
  MSSQL_SA_PASSWORD: ${MSSQL_SA_PASSWORD}
  MSSQL_PID: ${MSSQL_PID}
ports:
  - "1433:1433"
volumes:
  - navi_sqlserver_data:/var/opt/mssql
networks:
  - navi_tools_network

minio:
  image: minio/minio:latest
  container_name: navi-tools-minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ROOT_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
  ports:
    - "9000:9000"
    - "9001:9001"
  volumes:
    - navi_minio_data:/data
  networks:
    - navi_tools_network

volumes:
  navi_sqlserver_data:
  navi_minio_data:

networks:
  navi_tools_network:
    driver: bridge
EOF

```

6.8 Crear override para Mac Apple Silicon

```

Ejecuta:
if [ "$(uname -m)" = "arm64" ]; then
  cat > docker/docker-compose.mac.override.yml <<'EOF'
services:
  sqlserver:
    platform: linux/amd64
EOF

```

```

fi
Validar:
if [ -f docker/docker-compose.mac.override.yml ]; then
    cat docker/docker-compose.mac.override.yml
fi

```

6.9 Levantar SQL Server y MinIO en Mac

Ejecuta:

```
open -a Docker
```

```

while ! docker info >/dev/null 2>&1; do
    echo "Esperando Docker..."
    sleep 5
done

```

Si es Apple Silicon:

```

docker compose --env-file docker/env/local.env \
    -f docker/docker-compose.yml \
    -f docker/docker-compose.mac.override.yml \
    up -d

```

Si es Intel:

```

docker compose --env-file docker/env/local.env \
    -f docker/docker-compose.yml \
    up -d

```

Verificar:

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
```

Debes ver:

```

navi-tools-sqlserver
navi-tools-minio

```

6.10 Esperar SQL Server en Mac

```
docker logs -f navi-tools-sqlserver
```

Cuando SQL Server esté listo, salir con:

```
Ctrl + C
```

Validar puerto:

```
nc -vz localhost 1433
```

6.11 Restaurar backup en Mac

Ejecuta:

```

cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
Activos

```

```

export MSSQL_SA_PASSWORD="$(grep '^MSSQL_SA_PASSWORD=' docker/
env/local.env | cut -d= -f2-)"

```



```
docker exec navi-tools-sqlserver mkdir -p /var/opt/mssql/backup
```

```
docker cp backups/NaviToolsAssetsDb.bak navi-tools-sqlserver:/var/opt/mssql/backup/NaviToolsAssetsDb.bak
```

```
SQLCMD="/opt/mssql-tools18/bin/sqlcmd"
```

```
if ! docker exec navi-tools-sqlserver test -x "$SQLCMD"; then
    SQLCMD="/opt/mssql-tools/bin/sqlcmd"
fi
```

```
docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD" navi-tools-sqlserver "$SQLCMD" \
    -S localhost \
    -U sa \
    -C \
    -Q "RESTORE FILELISTONLY FROM DISK = N'/var/opt/mssql/backup/NaviToolsAssetsDb.bak';"
```

Restaurar base:

```
docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD" navi-tools-sqlserver "$SQLCMD" \
    -S localhost \
    -U sa \
    -C \
    -Q "
```

```
IF DB_ID('NaviToolsAssetsDb') IS NOT NULL
BEGIN
```

```
    ALTER DATABASE [NaviToolsAssetsDb] SET SINGLE_USER WITH
    ROLLBACK IMMEDIATE;
END;
```

```
RESTORE DATABASE [NaviToolsAssetsDb]
FROM DISK = N'/var/opt/mssql/backup/NaviToolsAssetsDb.bak'
WITH REPLACE,
MOVE N'NaviToolsAssetsDb' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb.mdf',
MOVE N'NaviToolsAssetsDb_log' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb_log.ldf',
STATS = 10;
```

```
ALTER DATABASE [NaviToolsAssetsDb] SET MULTI_USER;
"
```

Validar restauración:

```

docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD" navi-tools-
sqlserver "$SQLCMD" \
  -S localhost \
  -U sa \
  -C \
  -d NaviToolsAssetsDb \
  -Q "
SELECT DB_NAME() AS BaseActual;
SELECT COUNT(*) AS Roles FROM dbo.AppRoles;
SELECT COUNT(*) AS Usuarios FROM dbo.AppUsers;
SELECT TOP 20 TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
ORDER BY TABLE_SCHEMA, TABLE_NAME;
"

```

6.12 Crear bucket de MinIO en Mac

Ejecuta:

```

export MINIO_ROOT_USER="$(grep '^MINIO_ROOT_USER=' docker/env/
local.env | cut -d= -f2-)"
export MINIO_ROOT_PASSWORD="$(grep '^MINIO_ROOT_PASSWORD='
docker/env/local.env | cut -d= -f2-)"
export MINIO_BUCKET="$(grep '^MINIO_BUCKET=' docker/env/
local.env | cut -d= -f2-)"

```

```

export NAVI_NETWORK="$(docker network ls --format '{{.Name}}' |
grep 'navi.*tools.*network' | head -n 1)"

```

```

docker run --rm --network "$NAVI_NETWORK" minio/mc \
  sh -c "
  mc alias set local http://navi-tools-minio:9000
  '$MINIO_ROOT_USER' '$MINIO_ROOT_PASSWORD' &&
  mc mb -p local/$MINIO_BUCKET || true &&
  mc ls local
  "

```

Abrir MinIO:

```
open http://localhost:9001
```

6.13 Restaurar documentos MinIO en Mac si existen

Si tienes respaldo en:

```
backups/minio/navi-tools-documents
```

ejecuta:

```

export MINIO_ROOT_USER="$(grep '^MINIO_ROOT_USER=' docker/env/
local.env | cut -d= -f2-)"
export MINIO_ROOT_PASSWORD="$(grep '^MINIO_ROOT_PASSWORD='

```

```
docker/env/local.env | cut -d= -f2-)"
export MINIO_BUCKET="$(grep '^MINIO_BUCKET=' docker/env/
local.env | cut -d= -f2-)"
export NAVI_NETWORK="$(docker network ls --format '{{.Name}}' |
grep 'navi.*tools.*network' | head -n 1)"
```

```
docker run --rm \
  --network "$NAVI_NETWORK" \
  -v "$(pwd)/backups/minio:/backup" \
  minio/mc \
  sh -c "
  mc alias set local http://navi-tools-minio:9000
'$MINIO_ROOT_USER' '$MINIO_ROOT_PASSWORD' &&
  mc mirror /backup/navi-tools-documents local/$MINIO_BUCKET
"
```

6.14 Compilar solución en Mac

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
Activos
```

```
dotnet restore Navitrans.ToolsAssets.Management.sln
```

```
dotnet build Navitrans.ToolsAssets.Management.sln -c Debug
```

6.15 Crear scripts de ejecución en Mac

```
mkdir -p scripts/mac
```

Script de variables Mac

```
cat > scripts/mac/load-env.sh <<'EOF'
#!/usr/bin/env bash
```

```
export ASPNETCORE_ENVIRONMENT="Development"
```

```
export MSSQL_SA_PASSWORD="$(grep '^MSSQL_SA_PASSWORD=' docker/
env/local.env | cut -d= -f2-)"
export MINIO_ROOT_USER="$(grep '^MINIO_ROOT_USER=' docker/env/
local.env | cut -d= -f2-)"
export MINIO_ROOT_PASSWORD="$(grep '^MINIO_ROOT_PASSWORD='
docker/env/local.env | cut -d= -f2-)"
export MINIO_BUCKET="$(grep '^MINIO_BUCKET=' docker/env/
local.env | cut -d= -f2-)"
```

```
export
ConnectionStrings__NaviToolsAssetsDb="Server=localhost,1433;Dat
abase=NaviToolsAssetsDb;User
```

```
Id=sa;Password=$MSSQL_SA_PASSWORD;TrustServerCertificate=True;"
```

```
export Minio__Endpoint="localhost:9000"  
export Minio__AccessKey="$MINIO_ROOT_USER"  
export Minio__SecretKey="$MINIO_ROOT_PASSWORD"  
export Minio__BucketName="$MINIO_BUCKET"  
export Minio__UseSsl="false"  
EOF
```

```
chmod +x scripts/mac/load-env.sh
```

Script API Mac

```
cat > scripts/mac/run-api.sh <<'EOF'  
#!/usr/bin/env bash  
set -e
```

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-  
Activos
```

```
source scripts/mac/load-env.sh
```

```
dotnet run \  
  --project src/Navi.ToolsAssets.Api/  
Navi.ToolsAssets.Api.csproj \  
  --urls "http://localhost:5218"  
EOF
```

```
chmod +x scripts/mac/run-api.sh
```

Script Admin Mac

```
cat > scripts/mac/run-admin.sh <<'EOF'  
#!/usr/bin/env bash  
set -e
```

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-  
Activos
```

```
dotnet run \  
  --project src/Navi.ToolsAssets.Admin/  
Navi.ToolsAssets.Admin.csproj \  
  --urls "http://localhost:5264"  
EOF
```

```
chmod +x scripts/mac/run-admin.sh
```

Script Mobile Mac

```
cat > scripts/mac/run-mobile.sh <<'EOF'
```

```
#!/usr/bin/env bash
set -e

cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
Activos

dotnet run \
  --project src/Navi.ToolsAssets.MobilePwa/
Navi.ToolsAssets.MobilePwa.csproj \
  --urls "http://localhost:5285"
EOF

chmod +x scripts/mac/run-mobile.sh
```

6.16 Ejecutar proyecto en Mac

Terminal 1 — API:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
Activos
./scripts/mac/run-api.sh
```

Terminal 2 — Admin:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
Activos
./scripts/mac/run-admin.sh
```

Terminal 3 — Mobile:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-
Activos
./scripts/mac/run-mobile.sh
```

Abrir:

```
open http://localhost:5218/swagger
open http://localhost:5264
open http://localhost:5285
open http://localhost:9001
```

7. PARTE C — Instalación en Windows

7.1 Instalar herramientas en Windows

Abrir PowerShell como administrador.

Instalar Git:

```
winget install --id Git.Git -e
```

Instalar Docker Desktop:

```
winget install --id Docker.DockerDesktop -e
```

Instalar .NET 8 SDK:

```
winget install --id Microsoft.DotNet.SDK.8 -e
```

Instalar Visual Studio Code:

```
winget install --id Microsoft.VisualStudioCode -e
Cerrar y abrir PowerShell nuevamente.
Validar:
git --version
docker --version
docker compose version
dotnet --list-sdks
dotnet --version
Instalar Entity Framework:
dotnet tool install --global dotnet-ef --version 8.0.20
Si ya existe:
dotnet tool update --global dotnet-ef --version 8.0.20
Validar:
dotnet ef --version
```

7.2 Clonar proyecto en Windows

```
Crear carpeta:
New-Item -ItemType Directory -Force -Path "C:\
\Proyecto_Navi\Projects" | Out-Null
cd "C:\Proyecto_Navi\Projects"
Clonar:
git clone PEGAR_URL_DEL_REPOSITORIO_GIT Herramientas-Activos
Entrar:
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
Verificar:
dir
git status
```

7.3 Copiar backup al proyecto en Windows

```
Crear carpeta:
New-Item -ItemType Directory -Force -Path ".\backups" | Out-
Null
Copiar backup:
Copy-Item "C:\RUTA\DONDE\TENGAS\NaviToolsAssetsDb.bak" ".
\backups\NaviToolsAssetsDb.bak" -Force
Validar:
Get-Item ".\backups\NaviToolsAssetsDb.bak"
```

7.4 Crear docker/env/local.env en Windows

```
Ejecuta:
New-Item -ItemType Directory -Force -Path ".\docker\env" | Out-
Null
```

@"

```
ACCEPT_EULA=Y
MSSQL_SA_PASSWORD=PEGAR_PASSWORD_SQL_LOCAL
MSSQL_PID=Developer
MSSQL_DB=NaviToolsAssetsDb

MINIO_ROOT_USER=PEGAR_USUARIO_MINIO_LOCAL
MINIO_ROOT_PASSWORD=PEGAR_PASSWORD_MINIO_LOCAL
MINIO_BUCKET=navi-tools-documents
"@ | Set-Content ".\docker\env\local.env" -Encoding UTF8
Validar:
Get-Content ".\docker\env\local.env"
```

7.5 Crear docker-compose.yml estándar en Windows

```
@
name: navi-tools-assets

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: navi-tools-sqlserver
    restart: unless-stopped
    environment:
      ACCEPT_EULA: `${ACCEPT_EULA}
      MSSQL_SA_PASSWORD: `${MSSQL_SA_PASSWORD}
      MSSQL_PID: `${MSSQL_PID}
    ports:
      - "1433:1433"
    volumes:
      - navi_sqlserver_data:/var/opt/mssql
    networks:
      - navi_tools_network

  minio:
    image: minio/minio:latest
    container_name: navi-tools-minio
    restart: unless-stopped
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: `${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: `${MINIO_ROOT_PASSWORD}
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
```

```

        - navi_minio_data:/data
networks:
    - navi_tools_network

volumes:
    navi_sqlserver_data:
    navi_minio_data:

networks:
    navi_tools_network:
        driver: bridge
"@ | Set-Content ".\docker\docker-compose.yml" -Encoding UTF8

```

7.6 Levantar SQL Server y MinIO en Windows

Abrir Docker Desktop.

Luego ejecutar:

```

docker compose --env-file ".\docker\env\local.env" `
    -f ".\docker\docker-compose.yml" `
    up -d

```

Verificar:

```

docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"

```

Debes ver:

```

navi-tools-sqlserver
navi-tools-minio

```

7.7 Restaurar backup en Windows

Ejecuta:

```

$SqlPass = (Get-Content ".\docker\env\local.env" | Where-Object
{ $_ -like "MSSQL_SA_PASSWORD=*" }) -replace
"^MSSQL_SA_PASSWORD=", ""

```

```

docker exec navi-tools-sqlserver mkdir -p /var/opt/mssql/backup

```

```

docker cp ".\backups\NaviToolsAssetsDb.bak" navi-tools-
sqlserver:/var/opt/mssql/backup/NaviToolsAssetsDb.bak

```

```

$SqlCmd = "/opt/mssql-tools18/bin/sqlcmd"
docker exec navi-tools-sqlserver test -x $SqlCmd

```

```

if ($LASTEXITCODE -ne 0) {
    $SqlCmd = "/opt/mssql-tools/bin/sqlcmd"
}

```

```

docker exec -e SQLCMDPASSWORD="$SqlPass" navi-tools-sqlserver

```



```

$SqlCmd `
    -S localhost `
    -U sa `
    -C `
    -Q "RESTORE FILELISTONLY FROM DISK = N'/var/opt/mssql/
backup/NaviToolsAssetsDb.bak';"
Restaurar base:
docker exec -e SQLCMDPASSWORD="$SqlPass" navi-tools-sqlserver
$SqlCmd `
    -S localhost `
    -U sa `
    -C `
    -Q "
IF DB_ID('NaviToolsAssetsDb') IS NOT NULL
BEGIN
    ALTER DATABASE [NaviToolsAssetsDb] SET SINGLE_USER WITH
ROLLBACK IMMEDIATE;
END;

RESTORE DATABASE [NaviToolsAssetsDb]
FROM DISK = N'/var/opt/mssql/backup/NaviToolsAssetsDb.bak'
WITH REPLACE,
MOVE N'NaviToolsAssetsDb' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb.mdf',
MOVE N'NaviToolsAssetsDb_log' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb_log.ldf',
STATS = 10;

ALTER DATABASE [NaviToolsAssetsDb] SET MULTI_USER;
"
Validar:
docker exec -e SQLCMDPASSWORD="$SqlPass" navi-tools-sqlserver
$SqlCmd `
    -S localhost `
    -U sa `
    -C `
    -d NaviToolsAssetsDb `
    -Q "
SELECT DB_NAME() AS BaseActual;
SELECT COUNT(*) AS Roles FROM dbo.AppRoles;
SELECT COUNT(*) AS Usuarios FROM dbo.AppUsers;
SELECT TOP 20 TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
ORDER BY TABLE_SCHEMA, TABLE_NAME;

```

"

7.8 Crear bucket MinIO en Windows

```
$MinioUser = (Get-Content ".\docker\env\local.env" | Where-Object { $_ -like "MINIO_ROOT_USER=*" }) -replace
"^MINIO_ROOT_USER=", ""
$MinioPass = (Get-Content ".\docker\env\local.env" | Where-Object { $_ -like "MINIO_ROOT_PASSWORD=*" }) -replace
"^MINIO_ROOT_PASSWORD=", ""
$MinioBucket = (Get-Content ".\docker\env\local.env" | Where-Object { $_ -like "MINIO_BUCKET=*" }) -replace
"^MINIO_BUCKET=", ""

$NaviNetwork = docker network ls --format "{{.Name}}" | Where-Object { $_ -match "navi.*tools.*network" } | Select-Object -First 1

docker run --rm --network $NaviNetwork minio/mc `
  sh -c "mc alias set local http://navi-tools-minio:9000
'$MinioUser' '$MinioPass' && mc mb -p local/$MinioBucket ||
true && mc ls local"
Abrir MinIO:
Start-Process "http://localhost:9001"
```

7.9 Compilar solución en Windows

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"

dotnet restore ".\Navitrans.ToolsAssets.Management.sln"

dotnet build ".\Navitrans.ToolsAssets.Management.sln" -c Debug
```

7.10 Crear scripts de ejecución en Windows

```
New-Item -ItemType Directory -Force -Path ".\scripts\windows" |
Out-Null
```

Script API Windows

```
@'
$ProjectRoot = "C:\Proyecto_Navi\Projects\Herramientas-Activos"
Set-Location $ProjectRoot

$SqlPass = (Get-Content ".\docker\env\local.env" | Where-Object { $_ -like "MSSQL_SA_PASSWORD=*" }) -replace
"^MSSQL_SA_PASSWORD=", ""
$MinioUser = (Get-Content ".\docker\env\local.env" | Where-
```

```
Object { $_ -like "MINIO_ROOT_USER=*" }) -replace
"^MINIO_ROOT_USER=", ""
$MinioPass = (Get-Content ".\docker\env\local.env" | Where-
Object { $_ -like "MINIO_ROOT_PASSWORD=*" }) -replace
"^MINIO_ROOT_PASSWORD=", ""
$MinioBucket = (Get-Content ".\docker\env\local.env" | Where-
Object { $_ -like "MINIO_BUCKET=*" }) -replace
"^MINIO_BUCKET=", ""
```

```
$env:ASPNETCORE_ENVIRONMENT = "Development"
$env:ASPNETCORE_URLS = "http://localhost:5218"
```

```
$env:ConnectionStrings__NaviToolsAssetsDb =
"Server=localhost,1433;Database=NaviToolsAssetsDb;User
Id=sa;Password=$SqlPass;TrustServerCertificate=True;"
```

```
$env:Minio__Endpoint = "localhost:9000"
$env:Minio__AccessKey = $MinioUser
$env:Minio__SecretKey = $MinioPass
$env:Minio__BucketName = $MinioBucket
$env:Minio__UseSsl = "false"
```

```
dotnet run --project ".
\src\Navi.ToolsAssets.Api\Navi.ToolsAssets.Api.csproj" --urls
"http://localhost:5218"
'@ | Set-Content ".\scripts\windows\run-api.ps1" -Encoding UTF8
```

Script Admin Windows

```
@'
$ProjectRoot = "C:\Proyecto_Navi\Projects\Herramientas-Activos"
Set-Location $ProjectRoot
```

```
dotnet run --project ".
\src\Navi.ToolsAssets.Admin\Navi.ToolsAssets.Admin.csproj" --
urls "http://localhost:5264"
'@ | Set-Content ".\scripts\windows\run-admin.ps1" -Encoding
UTF8
```

Script Mobile Windows

```
@'
$ProjectRoot = "C:\Proyecto_Navi\Projects\Herramientas-Activos"
Set-Location $ProjectRoot
```

```
dotnet run --project ".
\src\Navi.ToolsAssets.MobilePwa\Navi.ToolsAssets.MobilePwa.cspr
oj" --urls "http://localhost:5285"
```

```
'@ | Set-Content ".\scripts\windows\run-mobile.ps1" -Encoding UTF8
```

7.11 Ejecutar proyecto en Windows

Terminal 1 — API:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\run-api.ps1
```

Terminal 2 — Admin:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\run-admin.ps1
```

Terminal 3 — Mobile:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\run-mobile.ps1
```

Abrir:

```
Start-Process "http://localhost:5218/swagger"
```

```
Start-Process "http://localhost:5264"
```

```
Start-Process "http://localhost:5285"
```

```
Start-Process "http://localhost:9001"
```

8. Script completo para Mac

Si quieres dejar todo en un solo script para Mac, crea:

```
nano setup-navi-mac.sh
```

Pega:

```
#!/usr/bin/env bash
```

```
set -e
```

```
PROJECT_ROOT="/Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-Activos"
```

```
SQL_CONTAINER="navi-tools-sqlserver"
```

```
MINIO_CONTAINER="navi-tools-minio"
```

```
DB_NAME="NaviToolsAssetsDb"
```

```
BACKUP_LOCAL_PATH="backups/NaviToolsAssetsDb.bak"
```

```
BACKUP_CONTAINER_PATH="/var/opt/mssql/backup/NaviToolsAssetsDb.bak"
```

```
cd "$PROJECT_ROOT"

if [ ! -f "$BACKUP_LOCAL_PATH" ]; then
    echo "No existe $BACKUP_LOCAL_PATH"
    exit 1
fi

mkdir -p docker/env backups scripts/mac

if [ ! -f docker/env/local.env ]; then
    echo "Debes crear docker/env/local.env antes de continuar."
    exit 1
fi

cat > docker/docker-compose.yml <<'YML'
name: navi-tools-assets

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: navi-tools-sqlserver
    restart: unless-stopped
    environment:
      ACCEPT_EULA: ${ACCEPT_EULA}
      MSSQL_SA_PASSWORD: ${MSSQL_SA_PASSWORD}
      MSSQL_PID: ${MSSQL_PID}
    ports:
      - "1433:1433"
    volumes:
      - navi_sqlserver_data:/var/opt/mssql
    networks:
      - navi_tools_network

  minio:
    image: minio/minio:latest
    container_name: navi-tools-minio
    restart: unless-stopped
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    ports:
      - "9000:9000"
```

```
    - "9001:9001"
  volumes:
    - navi_minio_data:/data
  networks:
    - navi_tools_network
```

```
volumes:
  navi_sqlserver_data:
  navi_minio_data:
```

```
networks:
  navi_tools_network:
    driver: bridge
```

YML

```
if [ "$(uname -m)" = "arm64" ]; then
  cat > docker/docker-compose.mac.override.yml <<'YML'
```

```
services:
  sqlserver:
    platform: linux/amd64
```

YML

fi

```
open -a Docker || true
```

```
while ! docker info >/dev/null 2>&1; do
  echo "Esperando Docker..."
  sleep 5
done
```

```
if [ "$(uname -m)" = "arm64" ]; then
  docker compose --env-file docker/env/local.env \
    -f docker/docker-compose.yml \
    -f docker/docker-compose.mac.override.yml \
    up -d
```

else

```
  docker compose --env-file docker/env/local.env \
    -f docker/docker-compose.yml \
    up -d
```

fi

```
echo "Esperando SQL Server..."
sleep 20
```

```
export MSSQL_SA_PASSWORD="$(grep '^MSSQL_SA_PASSWORD=' docker/
env/local.env | cut -d= -f2-)"
```

```
SQLCMD="/opt/mssql-tools18/bin/sqlcmd"
```

```
if ! docker exec "$SQL_CONTAINER" test -x "$SQLCMD"; then
    SQLCMD="/opt/mssql-tools/bin/sqlcmd"
fi
```

```
for i in {1..60}; do
    if docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD"
"$SQL_CONTAINER" "$SQLCMD" \
        -S localhost \
        -U sa \
        -C \
        -Q "SELECT 1" >/dev/null 2>&1; then
        echo "SQL Server listo."
        break
    fi
    echo "Esperando SQL Server intento $i..."
    sleep 5
done
```

```
docker exec "$SQL_CONTAINER" mkdir -p /var/opt/mssql/backup
docker cp "$BACKUP_LOCAL_PATH" "$SQL_CONTAINER:
$BACKUP_CONTAINER_PATH"
```

```
docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD"
"$SQL_CONTAINER" "$SQLCMD" \
    -S localhost \
    -U sa \
    -C \
    -Q "
IF DB_ID('$DB_NAME') IS NOT NULL
BEGIN
    ALTER DATABASE [$DB_NAME] SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;
END;

RESTORE DATABASE [$DB_NAME]
FROM DISK = N'$BACKUP_CONTAINER_PATH'
WITH REPLACE,
MOVE N'NaviToolsAssetsDb' TO N'/var/opt/mssql/data/
```

```

NaviToolsAssetsDb.mdf',
MOVE N'NaviToolsAssetsDb_log' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb_log.ldf',
STATS = 10;

ALTER DATABASE [$DB_NAME] SET MULTI_USER;
"

export MINIO_ROOT_USER="$(grep '^MINIO_ROOT_USER=' docker/env/
local.env | cut -d= -f2-)"
export MINIO_ROOT_PASSWORD="$(grep '^MINIO_ROOT_PASSWORD='
docker/env/local.env | cut -d= -f2-)"
export MINIO_BUCKET="$(grep '^MINIO_BUCKET=' docker/env/
local.env | cut -d= -f2-)"
export NAVI_NETWORK="$(docker network ls --format '{{.Name}}' |
grep 'navi.*tools.*network' | head -n 1)"

docker run --rm --network "$NAVI_NETWORK" minio/mc \
  sh -c "
  mc alias set local http://navi-tools-minio:9000
'$MINIO_ROOT_USER' '$MINIO_ROOT_PASSWORD' &&
  mc mb -p local/$MINIO_BUCKET || true &&
  mc ls local
  "

dotnet restore Navitrans.ToolsAssets.Management.sln
dotnet build Navitrans.ToolsAssets.Management.sln -c Debug

echo
"=====
echo "NAVI listo en Mac"
echo "SQL Server: localhost,1433"
echo "MinIO: http://localhost:9001"
echo "API: ejecutar ./scripts/mac/run-api.sh"
echo "Admin: ejecutar ./scripts/mac/run-admin.sh"
echo "Mobile: ejecutar ./scripts/mac/run-mobile.sh"
echo
"=====
Guardar y ejecutar:
chmod +x setup-navi-mac.sh
./setup-navi-mac.sh

```

9. Script completo para Windows

Crear:


```
New-Item -ItemType Directory -Force -Path ".\scripts\windows" | Out-Null
notepad ".\scripts\windows\setup-navi-windows.ps1"
Pegar:
$ProjectRoot = "C:\Proyecto_Navi\Projects\Herramientas-Activos"
$SqlContainer = "navi-tools-sqlserver"
$MinioContainer = "navi-tools-minio"
$DbName = "NaviToolsAssetsDb"
$BackupLocalPath = ".\backups\NaviToolsAssetsDb.bak"
$BackupContainerPath = "/var/opt/mssql/backup/
NaviToolsAssetsDb.bak"
```

```
Set-Location $ProjectRoot
```

```
if (!(Test-Path $BackupLocalPath)) {
    Write-Host "No existe $BackupLocalPath" -ForegroundColor Red
    exit 1
}
```

```
New-Item -ItemType Directory -Force -Path ".\docker\env" | Out-Null
New-Item -ItemType Directory -Force -Path ".\backups" | Out-Null
New-Item -ItemType Directory -Force -Path ".\scripts\windows" | Out-Null
```

```
if (!(Test-Path ".\docker\env\local.env")) {
    Write-Host "Debes crear docker\env\local.env antes de continuar." -ForegroundColor Red
    exit 1
}
```

```
@"
```

```
name: navi-tools-assets
```

```
services:
```

```
  sqlserver:
```

```
    image: mcr.microsoft.com/mssql/server:2022-latest
```

```
    container_name: navi-tools-sqlserver
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      ACCEPT_EULA: `${ACCEPT_EULA}
```

```
      MSSQL_SA_PASSWORD: `${MSSQL_SA_PASSWORD}
```

```

    MSSQL_PID: `${MSSQL_PID}`
  ports:
    - "1433:1433"
  volumes:
    - navi_sqlserver_data:/var/opt/mssql
  networks:
    - navi_tools_network

minio:
  image: minio/minio:latest
  container_name: navi-tools-minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: `${MINIO_ROOT_USER}`
    MINIO_ROOT_PASSWORD: `${MINIO_ROOT_PASSWORD}`
  ports:
    - "9000:9000"
    - "9001:9001"
  volumes:
    - navi_minio_data:/data
  networks:
    - navi_tools_network

volumes:
  navi_sqlserver_data:
  navi_minio_data:

networks:
  navi_tools_network:
    driver: bridge
"@ | Set-Content ".\docker\docker-compose.yml" -Encoding UTF8

docker compose --env-file ".\docker\env\local.env" `
  -f ".\docker\docker-compose.yml" `
  up -d

Start-Sleep -Seconds 20

$SqlPass = (Get-Content ".\docker\env\local.env" | Where-Object
{ $_ -like "MSSQL_SA_PASSWORD=*" }) -replace
"^MSSQL_SA_PASSWORD=", ""

$SqlCmd = "/opt/mssql-tools18/bin/sqlcmd"

```

```

docker exec $SqlContainer test -x $SqlCmd

if ($LASTEXITCODE -ne 0) {
    $SqlCmd = "/opt/mssql-tools/bin/sqlcmd"
}

for ($i = 1; $i -le 60; $i++) {
    docker exec -e SQLCMDPASSWORD="$SqlPass" $SqlContainer
$SqlCmd `
    -S localhost `
    -U sa `
    -C `
    -Q "SELECT 1" | Out-Null

    if ($LASTEXITCODE -eq 0) {
        Write-Host "SQL Server listo." -ForegroundColor Green
        break
    }

    Write-Host "Esperando SQL Server intento $i..."
    Start-Sleep -Seconds 5
}

docker exec $SqlContainer mkdir -p /var/opt/mssql/backup
docker cp $BackupLocalPath "${SqlContainer}:
$BackupContainerPath"

docker exec -e SQLCMDPASSWORD="$SqlPass" $SqlContainer $SqlCmd
`
    -S localhost `
    -U sa `
    -C `
    -Q "
IF DB_ID('$DbName') IS NOT NULL
BEGIN
    ALTER DATABASE [$DbName] SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;
END;

RESTORE DATABASE [$DbName]
FROM DISK = N'$BackupContainerPath'
WITH REPLACE,
MOVE N'NaviToolsAssetsDb' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb.mdf',

```

```

MOVE N'NaviToolsAssetsDb_log' TO N'/var/opt/mssql/data/
NaviToolsAssetsDb_log.ldf',
STATS = 10;

ALTER DATABASE [$DbName] SET MULTI_USER;
"

$MinioUser = (Get-Content ".\docker\env\local.env" | Where-
Object { $_ -like "MINIO_ROOT_USER=*" }) -replace
"^MINIO_ROOT_USER=", ""
$MinioPass = (Get-Content ".\docker\env\local.env" | Where-
Object { $_ -like "MINIO_ROOT_PASSWORD=*" }) -replace
"^MINIO_ROOT_PASSWORD=", ""
$MinioBucket = (Get-Content ".\docker\env\local.env" | Where-
Object { $_ -like "MINIO_BUCKET=*" }) -replace
"^MINIO_BUCKET=", ""

$NaviNetwork = docker network ls --format "{{.Name}}" | Where-
Object { $_ -match "navi.*tools.*network" } | Select-Object
-First 1

docker run --rm --network $NaviNetwork minio/mc `
  sh -c "mc alias set local http://navi-tools-minio:9000
'$MinioUser' '$MinioPass' && mc mb -p local/$MinioBucket ||
true && mc ls local"

dotnet restore ".\Navitrans.ToolsAssets.Management.sln"
dotnet build ".\Navitrans.ToolsAssets.Management.sln" -c Debug

Write-Host
"=====
-ForegroundColor Green
Write-Host "NAVI listo en Windows" -ForegroundColor Green
Write-Host "SQL Server: localhost,1433"
Write-Host "MinIO: http://localhost:9001"
Write-Host "API: .\scripts\windows\run-api.ps1"
Write-Host "Admin: .\scripts\windows\run-admin.ps1"
Write-Host "Mobile: .\scripts\windows\run-mobile.ps1"
Write-Host
"=====
-ForegroundColor Green
Ejecutar:
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"

```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\setup-navi-windows.ps1
```

10. Comandos diarios en Mac

Cada día:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-Activos
```

```
git pull
```

```
if [ "$(uname -m)" = "arm64" ]; then
  docker compose --env-file docker/env/local.env \
    -f docker/docker-compose.yml \
    -f docker/docker-compose.mac.override.yml \
    up -d
else
```

```
  docker compose --env-file docker/env/local.env \
    -f docker/docker-compose.yml \
    up -d
fi
```

Terminal 1:

```
./scripts/mac/run-api.sh
```

Terminal 2:

```
./scripts/mac/run-admin.sh
```

Terminal 3:

```
./scripts/mac/run-mobile.sh
```

11. Comandos diarios en Windows

Cada día:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
git pull
```

```
docker compose --env-file ".\docker\env\local.env" `
  -f ".\docker\docker-compose.yml" `
  up -d
```

Terminal 1:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\windows\run-api.ps1
```

Terminal 2:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\windows\run-admin.ps1
```

Terminal 3:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\scripts\windows\run-mobile.ps1
```

12. Validaciones finales

12.1 Validar Docker

Mac o Windows:

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
```

Debe aparecer:

```
navi-tools-sqlserver
```

```
navi-tools-minio
```

12.2 Validar Swagger

Abrir:

```
http://localhost:5218/swagger
```

12.3 Validar Admin

Abrir:

```
http://localhost:5264
```

12.4 Validar Mobile

Abrir:

```
http://localhost:5285
```

12.5 Validar MinIO

Abrir:

```
http://localhost:9001
```

12.6 Validar base de datos

Mac:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-  
Activos
```

```
export MSSQL_SA_PASSWORD="$(grep '^MSSQL_SA_PASSWORD=' docker/  
env/local.env | cut -d= -f2-)"
```

```
docker exec -e SQLCMDPASSWORD="$MSSQL_SA_PASSWORD" navi-tools-  
sqlserver \  
  /opt/mssql-tools18/bin/sqlcmd \  
  -S localhost \  
  -U sa \  
  -C \  
  -d NaviToolsAssetsDb \  
  -i assetsdb.sql
```

```

-Q "SELECT DB_NAME() AS BaseActual; SELECT COUNT(*) AS
Usuarios FROM dbo.AppUsers;"
Windows:
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"

$SqlPass = (Get-Content ".\docker\env\local.env" | Where-Object
{ $_ -like "MSSQL_SA_PASSWORD=*" }) -replace
"^MSSQL_SA_PASSWORD=", ""

docker exec -e SQLCMDPASSWORD="$SqlPass" navi-tools-sqlserver `
/opt/mssql-tools18/bin/sqlcmd `
-S localhost `
-U sa `
-C `
-d NaviToolsAssetsDb `
-Q "SELECT DB_NAME() AS BaseActual; SELECT COUNT(*) AS
Usuarios FROM dbo.AppUsers;"

```

13. Checklist final

- [] Git instalado.
- [] Docker Desktop instalado.
- [] Docker corriendo.
- [] .NET 8 SDK instalado.
- [] dotnet-ef instalado.
- [] Proyecto clonado desde Git.
- [] Backup ubicado en backups/NaviToolsAssetsDb.bak.
- [] docker/env/local.env creado.
- [] docker/docker-compose.yml creado con puertos 1433, 9000 y 9001.
- [] En Mac Apple Silicon existe docker/docker-compose.mac.override.yml.
- [] SQL Server levantado.
- [] MinIO levantado.
- [] Backup restaurado.
- [] Bucket navi-tools-documents creado.
- [] Solución compila.
- [] API corre en http://localhost:5218.
- [] Swagger abre.
- [] Admin corre en http://localhost:5264.
- [] Mobile PWA corre en http://localhost:5285.
- [] Login validado.
- [] No se subió local.env a Git.
- [] No se subió .bak a Git.

14. Resumen de ejecución rápida

Mac

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-Activos
```

```
./scripts/mac/run-api.sh
```

En otra terminal:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-Activos
```

```
./scripts/mac/run-admin.sh
```

En otra terminal:

```
cd /Users/luu/Desktop/Proyecto_Navi/Projects/Herramientas-Activos
```

```
./scripts/mac/run-mobile.sh
```

Windows

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\run-api.ps1
```

En otra terminal:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\run-admin.ps1
```

En otra terminal:

```
cd "C:\Proyecto_Navi\Projects\Herramientas-Activos"
```

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\scripts\windows\run-mobile.ps1
```

15. URLs finales

Swagger API:

<http://localhost:5218/swagger>

Admin Web:

<http://localhost:5264>

Mobile PWA:

<http://localhost:5285>

MinIO Console:

<http://localhost:9001>

SQL Server:

localhost,1433